

Henallux – Université de Namur

Architecture des systèmes informatiques



AI149 – Sécurité des Applications

Rapport final



MASTER 1

RUELLE Thomas

DEMIR Erdem

CLAUS Gatien

MOMBAERTS Ciarán

Mr. Absil et Mr. Patz

Année académique 2025-2026

Table des matières

Introduction	1
1 Cahier des charges	2
2 Choix techniques	3
2.1 Authentification	3
2.2 Langage backend.....	5
2.3 Langage frontend	6
2.4 Base de données	7
2.5 Serveur web	8
2.6 Certifications	9
2.7 SIEM	10
2.8 Algorithmes de sécurité	12
3 Modélisations	13
3.1 Inscription d'un patient.....	13
3.2 Inscription d'un médecin	14
3.2.1 Génération du certificat	14
3.2.2 Authentification et création du matériel cryptographique	15
3.3 Flux de chiffrement d'un fichier	16
3.4 Appairage d'un nouvel appareil	17
3.4.1 Patient : Passkey lié au hardware.....	17
3.4.2 Médecin : Certificat X.509 portable	18
3.5 Création d'une relation et accès au dossier patient	19
3.6 Flux d'approbation d'une action médecin	20
Conclusion	21
Bibliographie	22
Table des illustrations	24

Introduction

La sécurisation des données médicales des personnes représente une importance fondamentale, surtout via la montée de la technologie et de l'informatique dans le domaine médicale. La sensibilité des informations échangées entre les patients et les médecins se doit d'être protégé pouvant ainsi couvrir la confidentialité, l'intégrité et l'authenticité des données. C'est dans ce contexte que s'inscrit le projet du cours Sécurité des Applications, permettant de concevoir et à implémenter une application web client/serveur sécurisée dédiée à la gestion de dossiers médicaux. Cette application met en relation deux acteurs : les patients et les médecins.

L'objectif principal de ce projet est de développer une solution dont la sécurité constitue l'élément central de l'architecture, en choisissant les bonnes techniques. L'application doit garantir la protection des données même dans le cas où le serveur serait compromis, celui-ci étant considéré comme une entité non fiable à laquelle aucune information sensible ne peut être confiée en clair. Les échanges doivent être chiffrés, l'authentification des utilisateurs doit être assuré et le serveur doit posséder des mécanismes de défense contre les attaques courantes, comme les attaques de déni de service, attaques par dictionnaire ou les injections.

Ce rapport détaille l'ensemble des choix réalisés dans le cadre de ce projet. Il débute par la présentation de l'architecture générale ainsi que les acteurs impliqués. Ensuite, une analyse technique permet de définir les technologies retenues, notamment le serveur web et la gestion des certificats. Les protocoles de sécurité choisis sont ensuite décrits en détail, expliquant l'enregistrement et l'authentification des utilisateurs, le contrôle d'accès aux dossiers médicaux ainsi que les mécanismes de protection des données. [1]

1 Cahier des charges

Cette section dresse la liste des fonctionnalités définies dans le cadre du projet du portail santé :

- Gestion de documents médicaux : Ajout, mise à jour et suppression définitive de fichiers par le patient dans son propre dossier de manière sécurisée.
- Consultation des dossiers médicaux sécurisée : Accès au dossier médical limité au patient et aux médecins qu'il a autorisés.
- Gestion des relations : Ajout ou suppression de médecins par le patient à sa liste.
- Validation explicite des actions : Possibilité pour le médecin d'initier un ajout/suppression de document ou une demande de mise en relation, soumis à l'approbation du patient.
- Authentification Zero-Knowledge : Connexion des patients une authentification zero-knowledge et authentification des médecins via certificats délivrés par une PKI locale.
- Chiffrement de bout-en-bout : Chiffrement des données à l'envoi afin de rendre le serveur aveugle sur ces données et limiter les risques liés à un administrateur malicieux.
- Gestion multi-appareils : Mécanisme de synchronisation des accès lors de l'ajout d'un nouvel appareil.
- Traçabilité forte : Journalisation avec de forte garantie d'intégrité exploitant des métadonnées pour la détection d'anomalies, en respectant les principes de segregation of duties.
- Protection contre les vulnérabilités web : Mise en place de sécurités contre les attaques courantes de l'OWASP.
- Automatisation du déploiement : Scripts garantissant l'installation, la compilation et l'exécution du projet.

2 Choix techniques

Cette section a pour objectif de décrire et d'expliquer les différents choix techniques réalisés afin de mener à bien ce projet. Chacun de ceux-ci sont issus de recherches et d'analyse, sur base des besoins de l'application établis par le client :

Pour chacun des comparatifs réalisés, une échelle numérique de 1 à 5 sera utilisée afin d'évaluer si la technologie répond au critère associé. De plus, les critères ne possédant pas tous la même importance, ils jouiront également d'une valeur numérique, de 1 à 5. L'évaluation de la pertinence des technologies se basera sur une moyenne pondérée des notes pour chaque critère (également pondérées) ramenée sur 100.

2.1 Authentification

Afin de répondre aux besoins du projet en matière de gestion de l'authentification, plusieurs solutions ont été envisagées :

- Keycloak : Développé par Red Hat, Keycloak est un gestionnaire d'identités et d'accès open-source reposant sur l'environnement d'exécution Java. Son moteur de flux d'authentification intègre les mécanismes nécessaires pour traiter les certificats X.509 émis par une PKI locale. Il permet également d'orchestrer ces étapes via une modélisation des flux, en y conditionnant des étapes de validation telles que l'authentification via le protocole WebAuthn. [2]
- Zitadel : Conçu en langage Go, Zitadel propose une architecture IAM orientée vers les environnements conteneurisés. Le cœur de son fonctionnement repose sur le paradigme de l'Event Sourcing : au lieu d'écraser les données lors d'une mise à jour, chaque action est enregistrée comme un événement unique. Concernant l'authentification par certificats, son architecture la délègue à des composants tiers plutôt que de les traiter dans son propre moteur. [3]
- À la main : Le développement d'un système d'authentification sur mesure implique l'implémentation complète de la couche IAM directement dans le code source de l'application. Cette approche requiert d'écrire et de maintenir les algorithmes de gestion des certificats X.509, d'intégrer les spécifications du standard FIDO2 et de gérer le cycle de vie des jetons de session. La sécurisation de l'infrastructure face aux standards d'attaques de l'OWASP, ainsi que la mise à jour continue face aux nouvelles vulnérabilités sont à la charge de l'équipe de développement. [4]

Afin de les évaluer, une série de critères ont été évalués :

- Intégration à une PKI : les médecins doivent obligatoirement s'authentifier par des certificats délivrés par une PKI locale. Il faut donc que la solution supporte les authentifications par certificats x509 ;
- Support de WebAuthn/FIDO2 : afin d'assurer une authentification zero-knowledge pour les patients, la technologie WebAuthn a été choisie. Il faut donc que la solution supporte WebAuthn ;
- Traçabilité : le projet nécessite une traçabilité forte, cela implique donc une intégration de la solution avec le système de gestion des logs. Il faut que la solution permette l'envoi de logs, en particulier ceux relatifs à la création de compte et aux authentifications ;
- Efficience : la solution doit présenter un rapport optimal entre la sécurisation des flux et la consommation des ressources système ;
- Documentation : la solution doit disposer d'une documentation officielle exhaustive et d'une communauté active facilitant l'intégration et la résolution de bugs de configuration ;
- Robustesse aux vulnérabilités : la sécurité étant au centre de ce projet, il est essentiel que la solution soit maintenue à jour sans pour autant être victime de CVE connues ;
- Courbe d'apprentissage : la solution choisie doit être facile d'utilisation afin que les équipes de développement puisse le maintenir après livraison du projet ;

Les comparatifs sont centralisés dans le tableau suivant :

Critères d'évaluation	Poids	Keycloak		Zitadel		A la main	
Intégration PKI	5	5	25	3	15	1	5
WebAuthn	5	5	25	5	25	1	5
Traçabilité	4	4	16	5	20	1	4
Efficience	4	3	12	5	20	1	4
Documentation	4	5	20	3	12	1	4
Robustesse aux vulnérabilités	4	4	16	4	16	1	4
Courbe apprentissage	3	5	15	3	9	1	3
Somme non-pondérée	35	31		28		7	
Somme pondérée	145	129		117		29	
Total /100	//	88,96%		80,69%		20,00%	

[5], [6], [7]

En conclusion, le choix s'est porté sur Keycloak. Si Zitadel se démarque par une efficience et une traçabilité légèrement supérieure, Keycloak creuse un écart décisif sur les critères les plus critiques du cahier des charges : la gestion native des certificats X.509 (PKI) et l'exhaustivité de sa documentation.

Enfin, la solution d'un développement à la main est écartée par le risque de vulnérabilités et la complexité d'implémentation. Ceci entre en totale contradiction avec les besoins du projet.

2.2 Langage backend

La mise en place du portail de santé sera divisée en une partie web, le frontend, ainsi qu'une partie applicative, le backend. Ce backend aura la responsabilité de faire les redirections entre le frontend et Keycloak pour l'authentification ainsi que la gestion des dossiers patients. Plusieurs solutions ont été envisagées :

- Python (Django): Django repose sur une architecture de type 'batteries included' fournissant un ensemble complet de composants logiciels natifs. Son ORM intégré facilite la modélisation des structures de base de données nécessaires au stockage du matériel cryptographique. Toutefois, sa conception monolithique et son modèle d'exécution synchrone induisent une empreinte mémoire et des temps de traitement plus lourds par rapport aux micro-frameworks dédiés aux API [8]
- Stack Python (FastAPI/SQLAlchemy/Pydantic): FastAPI est un micro-framework orienté API, construit sur le standard d'exécution asynchrone ASGI. Son architecture de sécurité s'appuie sur la bibliothèque Pydantic pour la sérialisation et la validation des données entrantes. Cette mécanique impose un typage strict et une vérification structurelle des requêtes HTTP. L'outil génère dynamiquement la documentation OpenAPI à partir du code, mais nécessite l'intégration manuelle de bibliothèques tierces, telles que SQLAlchemy, pour assurer la persistance des données.[9]
- Express.js (Node.js). Express.js est une infrastructure d'applications web asynchrone fonctionnant sur l'environnement d'exécution Node.js. Bien que l'usage de TypeScript permette d'ajouter une vérification de type statique à la compilation, le framework au moment de l'exécution est structurellement minimaliste. Il n'embarque aucune validation stricte des données, d'ORM ou de système de journalisation. L'intégration des contraintes de sécurité implique l'utilisation de dépendances externes. [10]

Afin de les évaluer, une série de critères ont été évalués :

- Intégration à Keycloak : puisque l'authentification est déléguée à Keycloak, il faut que la solution choisie puisse s'intégrer avec facilement et de manière sécurisée.
- Validation des entrées : le portail de santé permet à l'utilisateur de rentrer des informations qui seront transmises au backend. Il faut donc que la solution soit capable de réaliser une validation stricte de ses entrées pour éviter les actions malveillantes.
- Traçabilité : le projet nécessite une traçabilité forte, cela implique donc une intégration de la solution avec le système de gestion des logs. Il faut que la solution permette l'envoi de ces logs.
- Efficience : la solution doit présenter une architecture d'exécution optimisée afin de soutenir la charge du routage et des opérations de validation sans introduire de latence préjudiciable.
- Documentation : la solution doit disposer d'une documentation officielle exhaustive et d'une communauté active facilitant l'intégration et la résolution de bugs de configuration ;
- Robustesse aux vulnérabilités : la sécurité étant au centre de ce projet, il est essentiel que la solution soit maintenue à jour sans pour autant être victime de CVE connues ;
- Courbe d'apprentissage : la solution choisie doit être facile d'utilisation afin que les équipes de développement puisse le maintenir après livraison du projet ;

Les comparatifs sont centralisés dans le tableau suivant :

Critères d'évaluation	Poids	Stack Python		Django		Express.JS	
Intégration Keycloak	5	4	20	4	20	3	15
Validation	5	5	25	4	20	3	15
Traçabilité	5	4	20	4	20	3	15
Efficience	4	4	16	3	12	5	20
Courbe apprentissage	4	5	20	3	12	4	16
Robustesse aux vulnérabilités	4	5	20	4	16	2	8
Documentation	3	5	15	5	15	2	6
Somme non-pondérée	35	32		27		22	
Somme pondérée	150	136		115		95	
Total /100	//	90,67%		76,67%		63,33%	

[11], [12], [13], [14]

En conclusion, le choix s'est porté sur la stack Python (90,67%). Si Express.js présente un léger avantage en matière d'efficience pure, il est lourdement pénalisé par son manque de validation native et sa vulnérabilité structurelle sans ajout de dépendances tierces. À l'inverse, Django (72,67%) offre un cadre sécurisant mais souffre d'une courbe d'apprentissage plus élevée liée à son architecture monolithique. La stack FastAPI s'impose donc comme le compromis idéal : elle garantit une sécurité by design et une intégration fluide avec l'infrastructure IAM, tout en maintenant une excellente maintenabilité grâce à sa documentation auto-générée.

2.3 Langage frontend

Le choix de la technologie utilisée pour le serveur web repose sur deux critères principaux. Le premier est l'accès à la Web API, indispensable pour l'ensemble des opérations de chiffrement et de déchiffrement des fichiers ainsi que pour la gestion des clés cryptographiques. Le second est la volonté d'éviter les frameworks nécessitant des échanges permanents entre le navigateur et le serveur web pour le rendu ou l'exécution de certaines fonctionnalités, comme c'est le cas de certains usages de Next.js.[15]

Après analyse des différentes options disponibles, le choix s'est porté sur Vite, un outil de développement moderne s'appuyant sur la bibliothèque React. D'autres solutions ont également été envisagées, notamment le développement en JavaScript natif ou l'utilisation de frameworks plus légers tels que Svelte. Bien que ces alternatives présentent certains avantages en termes de performances ou de simplicité, l'écosystème mature de React, la disponibilité de nombreuses bibliothèques tierces, ainsi que l'expérience acquise par l'équipe sur cette technologie ont constitué des arguments déterminants.[16]

Au regard du rapport entre les risques techniques, les gains de productivité et la rapidité de développement attendue, React associé à Vite est apparu comme la solution la plus adaptée pour répondre aux besoins du projet tout en limitant les incertitudes liées à l'adoption d'une technologie moins éprouvée au sein de l'équipe.

2.4 Base de données

La base de données doit stocker des données particulièrement sensibles comme des clés cryptographiques au format JWK, KEK chiffrés et métadonnées des fichiers médicaux. Le choix du moteur a donc été guidé avant tout par des critères de sécurité.

Pour faire ce choix, on a mis en concurrence plusieurs choix comme PostgreSQL, MySQL et MongoDB et la comparaison s'est appuyée sur quatre critères qui correspondent à nos vrais besoins sur le projet

- **Robustesse** : ce que dit l'historique des CVE du moteur, et surtout la vitesse à laquelle les failles sont corrigées une fois découvertes.
- **Contrôle d'accès** : jusqu'où on peut descendre pour restreindre l'accès aux données. L'idéal, pour nous, c'est de pouvoir cloisonner ligne par ligne.
- **Support JSON / JWK** : à quel point c'est simple et efficace de stocker nos clés cryptographiques au format JWK.
- **Intégration FastAPI / SQLAlchemy** : est-ce que ça s'intègre directement avec l'ORM et le framework qu'on a déjà retenus côté backend, ou est-ce qu'il faut bricoler.

Critères	PostgreSQL	MySQL	MongoDB
Robustesse	Solide, corrigé vite, modèle éprouvé	Correct, mais plus de CVE et des configs par défaut trop ouvertes	Plus exposé, surtout aux injections NoSQL
Contrôle d'accès	RLS natif : le cloisonnement par ligne tient même si l'appli a un bug.	Des rôles, mais rien au niveau ligne.	Du RBAC, mais grossier, pas d'équivalent au RLS.
Support JSON / JWK	JSONB natif, rapide, indexable. Nickel pour les JWK.	Géré, mais moins abouti.	Pensé pour le JSON, top pour les JWK — mais le non-relationnel pose souci ailleurs.
Intégration FastAPI / SQLAlchemy	Tout natif, rien à modifier	Ça marche, mais quelques réglages manuels	Pas géré par SQLAlchemy : faudrait un ODM tiers

[17], [18], [19]

Le choix s'est naturellement porté sur PostgreSQL. La fonctionnalité Row-Level Security a été déterminante parce qu'elle garantit au niveau du moteur que chaque utilisateur n'accède qu'à ses propres données, ce qui constitue une couche de protection solide même en cas de faille applicative. En plus de ça son intégration native avec SQLAlchemy et son support JSONB en font la solution la plus cohérente avec le reste de la stack technique.

2.5 Serveur web

Afin de rendre le portail santé accessible aux utilisateurs, il doit être déployé sur un serveur web. Ce dernier aura la charge de terminer les connexions HTTPS ainsi que de jouer le rôle de reverse proxy, pour les appels aux backend. Plusieurs solutions ont été envisagées :

- Nginx : permet d'effectuer plusieurs choses. Tout d'abord, sa fonctionnalité principale est d'être un serveur Web HTTP. Néanmoins, il est capable d'effectuer d'autres tâches comme devenir un reverse proxy, un cache de contenu, un load balancer et un serveur proxy. Il a été développé par Igor Sysoev et est sous licence BSD. [20]
- Apache : est un serveur HTTP open-source destiné aux systèmes d'exploitation comme UNIX et Windows. Son objectif est de fournir un serveur efficace, sécurisé et extensible, en respectant les standards HTTP. [21]

Afin de les évaluer, une série de critères ont été évalués :

- Architecture et consommation : évalue la gestion des requêtes et l'utilisation des ressources serveur.
- Configuration : évalue la flexibilité au niveau configuration, ainsi que le support du reverse proxy.
- Sécurité : évalue la résistance aux différents types d'attaques.
- Distribution des contenus : évalue les performances de l'envoi des contenus statiques et dynamiques.

Les comparatifs sont centralisés dans le tableau suivant :

Critères d'évaluation	Nginx	Apache
Architecture et consommation	Gère les requêtes de façon plus algorithmique et traite les demandes de manière à économiser les ressources	Les requêtes sont gérées de manière à être traitées par processus ou Thread séparé. Le fait de créer un processus pour chaque connexion peut entraîner une perte de performances et de ressources
Configuration	La configuration se fait de manière centralisée, ne pouvant pas proposer une configuration basée sur les répertoires. De plus, il permet d'effectuer du reverse Proxy	La configuration se fait de manière décentralisée, permettant donc une configuration basée sur des répertoires (.htaccess)
Sécurité	Résiste mieux contre les attaques DoS/DDoS grâce à sa gestion asynchrone	Repose sur des modules externes pour posséder une protection
Distribution des contenus	Les fichiers statiques (comme images, CSS et HTML) sont livrés plus rapidement. Niveau dynamique, si on donne à Nginx une bonne configuration, il assurera une bonne stabilité et performance.	Plus lent que Nginx au niveau du contenu statique, notamment à cause de son architecture. Niveau dynamique, il interprète le code via des modules intégrés. Il est plus simple à mettre en place niveau dynamique.

[22], [23]

Le choix de la technologie ici est Nginx car il apporte directement toutes les fonctionnalités avec lui, en apportant avec une bonne sécurité au niveau DoS/DDoS et possède une architecture optimale afin d'économiser les ressources du serveur.

2.6 Certifications

Afin d'apporter une sécurité au niveau du serveur WEB, passant ainsi vers HTTPS. L'utilisation de Let's Encrypt avait été sélectionnée. Let's Encrypt est une autorité de certification fiable et reconnu sur la plupart des appareils. Il a la réputation de fournir un bon niveau de protection, tout en proposant ses certificats gratuitement.

Cependant, une contrainte technique est venue changer le choix de cette solution : l'environnement de développement repose exclusivement sur des adresses IP privées. Le problème étant que Let's Encrypt nécessite un nom de domaine public accessible pour émettre un certificat. Il a donc fallu choisir une autre solution et celle-ci serait d'utiliser une PKI locale, permettant donc de générer et gérer des certificats auto-signés au sein de l'environnement interne.

A titre d'analyse, les avantages et inconvénients de Let's Encrypt ont tout de même été évalués, servant de comparaison utile si l'environnement de développement se voit déployer vers un environnement de production.

Avantages	Inconvénients
C'est gratuit et surtout, reconnu comme autorité de certification fiable. Elle possède la réputation nécessaire pour fournir une protection adéquate.	Validation de domaine (DV) uniquement. Aucune validation d'organisation (OV) ou étendue (EV) possible.
Permet de renforcer la sécurité.	Validité courte à 90 jours.
Améliore le référencement (SEO) en répondant aux exigences des moteurs de recherche.	A éviter pour les paiements en ligne.
Renouvellement automatisé, réduisant les interventions manuelles à l'échéance.	//
Affirme l'identité de base du site.	//

Ici, Let's Encrypt reste un bon choix d'utilisation, surtout qu'elle est reconnue et offre une bonne sécurité et un bon référencement. Pour notre utilisation, les paiements en ligne ne sont pas nécessaires donc l'inconvénient ici n'est pas un problème pour le bon développement du projet. [24]

2.7 SIEM

Pour assurer la traçabilité et l'audit, il est indispensable de mettre en place un système de centralisation des logs robuste. Nous avons étudié plusieurs solutions comme Splunk, Wazuh et la suite ELK et nous les avons comparées avant de faire notre choix définitif.

Pour ce faire, les critères suivants ont été évalués :

- Modèle et coût : nature de la licence (propriétaire ou open-source) et impact sur la liberté d'utilisation et d'audit du code
- Cœur de métier : adéquation entre la vocation principale de la solution et le besoin réel du projet, à savoir la centralisation de logs applicatifs
- Méthode d'ingestion : manière dont les logs sont collectés et transmis à la solution, et degré de contrôle offert sur ce pipeline
- Flexibilité des logs : capacité à ingérer et traiter des logs sur mesure contenant des données métiers spécifiques au projet
- Sécurité : garanties offertes sur l'intégrité et l'immuabilité des logs stockés, notamment en cas de compromission du backend

Les comparatifs sont centralisés dans le tableau suivant :

Critères d'évaluation	Splunk	Wazuh	Suite ELK
Modèle et coût	Propriétaire, approche "boîte noire", coût de licence prohibitif	Open-source et gratuit	Open-source, didactique et gratuit
Cœur de métier	SIEM global en mode clé en main pour une entreprise	Solution HIDS qui est axée sur la détection d'intrusions sur les endpoints	Puits de logs applicatifs sur mesure et traitement de données brutes
Méthode d'ingestion	Utilise des agents propriétaires lourds	Nécessite le déploiement d'agents préconfigurés sur les machines	Contrôle total via Logstash qui reçoit du JSON direct, sans agent externe complexe
Flexibilité des logs	C'est bien mais ça dépendante de l'écosystème éditeur	Trop rigide, orientée vers les alertes de sécurité système préexistantes	Flexibilité absolue permettant de vérifier des données métiers
Sécurité	Intégrée mais complexe à auditer (code fermé)	S'appuie sur Elasticsearch pour son propre stockage	Gestion RBAC qui est native permettant de restreindre Logstash à la création

[25], [26], [27]

Dans un premier temps, des solutions propriétaires comme Splunk ont été écartées. Bien qu'extrêmement puissantes et complètes, elles représentent un coût de licence prohibitif et une approche "boîte noire" qui ne correspond pas à la demande open-source et didactique de notre projet.

Notre principal besoin ce n'est pas la surveillance du système d'exploitation des machines c'est plutôt l'ingestion, le traitement et la sécurisation de logs purement applicatifs générés par notre backend. Nous avons donc décidé de partir directement sur la stack ELK brute. Ce choix s'impose comme le plus pertinent et le plus sécurisé pour notre cas d'usage précis, pour plusieurs raisons :

- Contrôle total du pipeline d'ingestion : En utilisant Logstash de manière autonome, il est possible de créer un pipeline http/HTTPS qui reçoit directement nos logs applicatifs au format JSON. Cela permet de ne dépendre d'aucun agent externe complexe à configurer.
- Implémentation de l'immuabilité : C'est l'argument de sécurité majeur de notre architecture. ELK nous donne un accès très granulaire aux rôles ce qui permet de configurer un utilisateur de service restreint qui ne possède que le privilège `create_doc` sur les index d'audit. Ainsi, Logstash peut uniquement ajouter de nouveaux événements. Même en cas de compromission totale de notre backend ou de l'API web, un attaquant n'a techniquement pas les droits nécessaires au niveau d'Elasticsearch pour modifier, altérer ou supprimer ses traces.
- Vérification cryptographique sur mesure : Parce que la flexibilité de Logstash permet de traiter les preuves d'intégrité et d'appliquer des filtres stricts et si un log arrive sans numéro de séquence ou sans hash valide, il est immédiatement tagué avec une alerte de sécurité avant même d'être stocké.

En résumé, là où une solution clé en main comme Wazuh aurait été trop rigide et orientée système, ELK permet de construire un puits de logs applicatif sur mesure, verrouillé en écriture seule, garantissant ainsi l'intégrité absolue des pistes d'audit exigée par ce projet

2.8 Algorithmes de sécurité

Au niveau des algorithmes de sécurité, plusieurs ont été utilisés lors de la conception de ce projet : RSA-OAEP-4096, AES-GCM-256 et l'utilisation, Key Wrapping et WebAuthn.

RSA-OAEP est un schéma de chiffrement asymétrique qui combine l'algorithme RSA avec un mécanisme de padding appelé OEAP. De base, RSA, à lui tout seul, est déterministe, c'est-à-dire que chiffrer deux fois le même message avec la même clé produit le même résultat. En utilisant l'OAEP (Optimal Asymmetric Encryption Padding), ça va permettre de corriger ce problème (le déterministe) en ajoutant une étape de transformation avant le chiffrement, permettant ainsi que le résultat soit toujours différent. De plus, l'utilisation de 4096 bits permet d'avoir une protection supplémentaire contre les attaques par brute-force. Son utilisation est purement utile pour chiffrer des petites données ou directement des clés symétriques. [28], [29], [30]

L'utilisation d'AES-GCM-256 n'est pas anodin non plus. Il permet de garantir la confidentialité des données et leurs intégrités. Une bonne logique a adopté puisque les données traitées sont des données extrêmement sensibles. Avoir du 256 bits offre une robustesse contre les attaques actuelles. Il est également très rapide grâce à l'accélération matérielle intégrée dans les processeurs modernes.

Dans le projet, il existe aussi un mécanisme de « Key Wrapping ». Celui-ci consiste à chiffrer une clé à l'aide d'une autre clé maîtresse. Cette technique permet protéger les clés sensibles en évitant leur stockage ou leur transmission en clair. Ça permet de renforcer la sécurité du système en assurant une confidentialité supplémentaire au sein du projet.[31], [32], [33]

L'utilisation de WebAuthn est aussi un atout puisque celui-ci permet une authentification web ultra-sécurisée et sans mot de passe en utilisant la cryptographie asymétrique. Le système remplace directement les mots de passe par des facteurs biométriques ou des clés matérielles. Son utilisation, au sein du projet, permettrait de réduire directement les risques liés au vol de mots de passe, au phishing ou autres types d'attaques. [34]

3 Modélisations

3.1 Inscription d'un patient

Le diagramme de séquence suivant décrit le flux d'inscription d'un patient dans le portail de santé. L'identité de l'utilisateur est d'abord gérée par Keycloak, qui prend en charge l'authentification zero-knowledge via WebAuthn, avant d'envoyer un JWT au client. Ensuite, le navigateur génère les clés cryptographiques du patient : une paire de clés asymétriques RSA-OAEP de 4096 bits (la clé publique PUK_pat et la clé privée PRK_pat) ainsi qu'une clé symétrique AES-GCM de 256 bits dédiés au dossier médical (KEK_folder). La clé privée est stockée dans le stockage local sécurisé du navigateur.

Enfin, la clé de dossier est chiffrée à l'aide de la clé publique générée. Le navigateur transmet la clé publique en clair (format JWK) et la clé de dossier chiffrée (Base64) au backend via deux appels distincts orientés « appareil » (/keys/register_device et /keys/store_kek). Le backend valide l'opération via la signature du JWT avant d'enregistrer ces données dans la base de données. Enfin, le dossier médical du patient est créé sur le disque du serveur, portant comme nom son user_id donné par Keycloak.

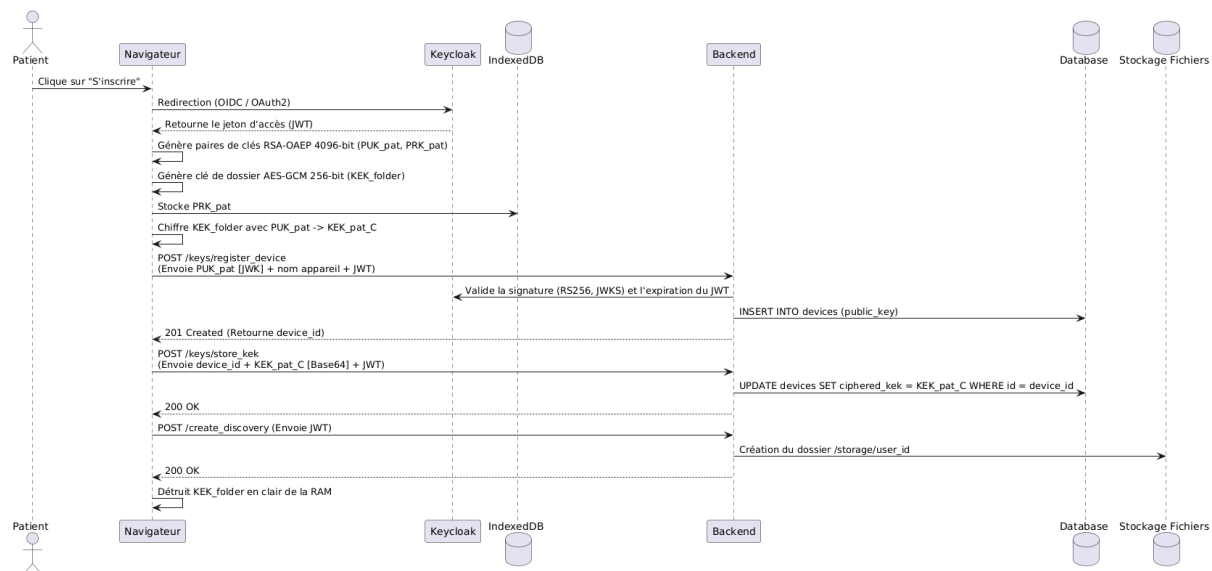


Figure 3-1 : Diagramme de séquence - Inscription patient

3.2 Inscription d'un médecin

3.2.1 Génération du certificat

Le diagramme de séquence modélise le flux d'enrôlement des médecins. Celui-ci commence par la saisie des informations et la génération d'une paire de clés asymétriques au sein du navigateur. Le client crée une requête de signature de certificat (CSR) et la transmet à la route POST /auth/register_doctor. Dès réception, le backend procède à une validation en interrogeant Keycloak afin de prévenir toute collision d'identifiants. Si les prérequis sont satisfaits, l'API dépose le fichier CSR sur un volume persistant partagé avec la PKI. Parallèlement, un processus au sein du conteneur PKI détecte l'apparition du CSR. La CA signe cryptographiquement la requête et génère un fichier temporaire (.crt.tmp). Afin de prévenir les conditions de concurrence en volume partagé, la PKI exécute un renommage atomique du fichier vers son extension finale (.crt), signalant au backend que le certificat est prêt. Dans le même temps, le processus PKI supprime le CSR initial.

L'API orchestre la création du compte utilisateur dans Keycloak puis récupère la chaîne de confiance de la PKI. Le backend supprime les fichiers sensibles avant de répondre au client. La réponse contient le certificat émis et la chaîne de confiance vers le navigateur. Le flux se finit par l'assemblage du certificat du médecin et de la clé privée, formant un certificat PKCS#12, qui est ensuite téléchargé.

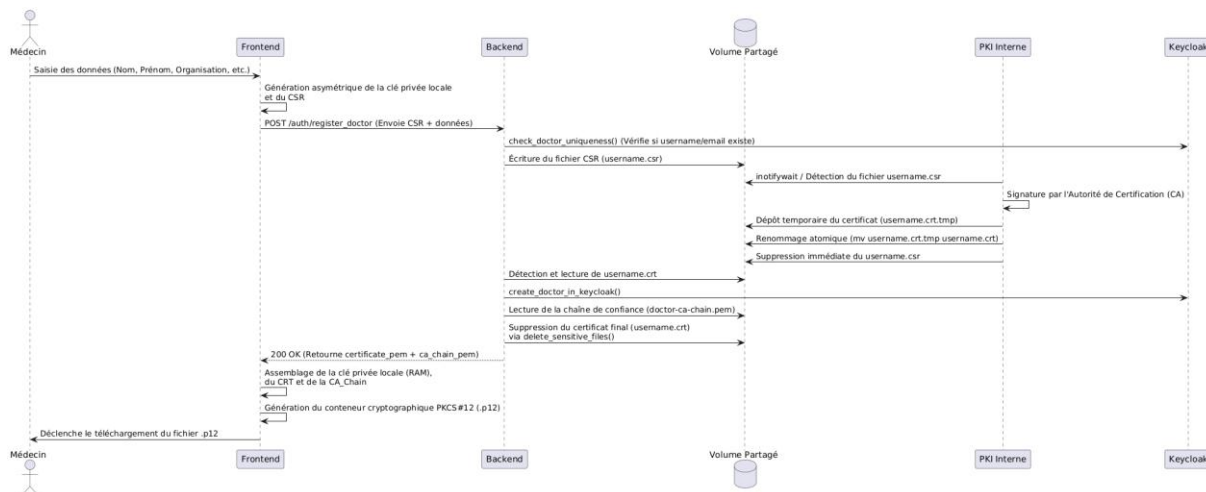


Figure 3-2 : Diagramme de séquence - Génération certificats

3.2.2 Authentification et création du matériel cryptographique

Le diagramme de séquence modélise le processus d' enrôlement initial d'un médecin. Le flux commence par une authentification par défi-réponse (Challenge-Response) basée sur le certificat X.509 du médecin. Le client signe cryptographiquement un défi généré par le serveur pour prouver son identité, permettant au backend de valider le flux et d'autoriser la redirection vers Keycloak. Celui-ci devra également créer son authentification WebAuthn avant de recevoir un jeton d'accès (JWT) sécurisé. Ensuite, le navigateur génère une première paire de clés asymétriques RSA-OAEP de 4096 bits dédiés à l'appareil. La clé privée est immédiatement stockée au sein du stockage IndexedDB, tandis que sa clé publique est transmise au backend via la route POST /keys/register_device afin d'enregistrer l'appareil.

Une fois l'appareil enregistré, le client génère la paire de clés du médecin, au format RSA-OAEP 4096 bits. Le client exécute un 'wrapping' de la clé privée en utilisant la clé publique de l'appareil. Cette opération lie la clé du médecin à l'appareil utilisé. Deux appels à l'API permettront le stockage sécurisé de ces clés sur le backend : l'enveloppe chiffrée est transmise à la route POST /keys/store_kek pour être associée à l'appareil dans la base de données. Ensuite, la clé publique est envoyée à la route POST /auth/store_public_mek pour être associée au profil de l'utilisateur.

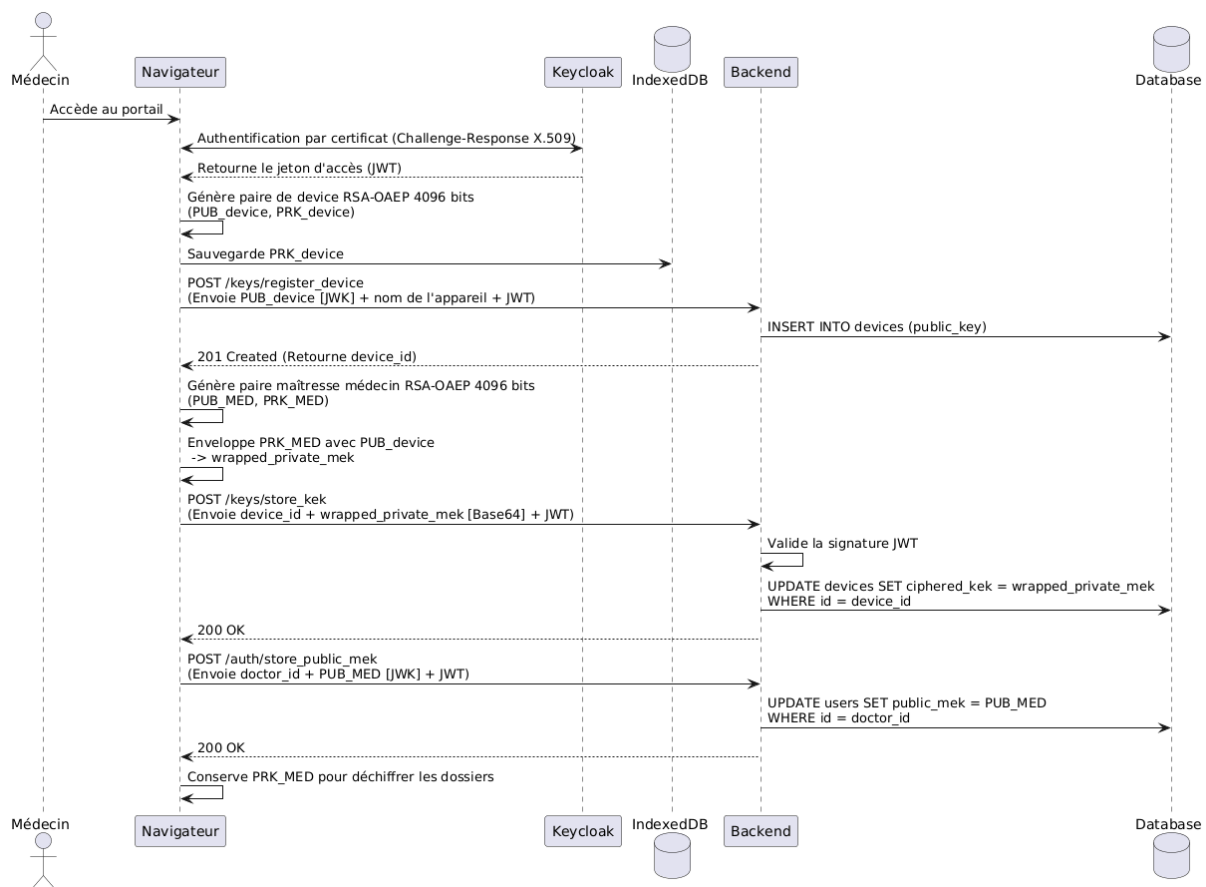


Figure 3-3 : Diagramme de séquence - Inscription médecin

3.3 Flux de chiffrement d'un fichier

Le diagramme de séquence modélise le flux de dépôt d'un document médical. L'opération débute côté client lorsque le patient sélectionne un fichier. Le navigateur génère une clé symétrique éphémère AES-GCM de 256 bits (DEK_file) ainsi que ses vecteurs d'initialisation (IVs). Cette clé permet de chiffrer indépendamment le contenu du fichier, son nom et la date. Pour sécuriser cette clé éphémère, le client procède à un enveloppement cryptographique (Key Wrapping) en la chiffrant avec la clé de dossier du patient (KEK_folder), préalablement chargée en mémoire. L'ensemble contenant la clé enveloppée et les IVs est ensuite formaté en une structure JSON encodée en Base64 (DEK_wrapped).

L'envoi vers le backend s'effectue via la route POST /files/upload_file. Le backend valide la signature du JWT pour authentifier et identifier l'utilisateur. Il écrit le fichier chiffré dans le dossier médical du patient, déduit à partir de son user_id. Les métadonnées chiffrées sont insérées dans la base de données, sans y stocker de chemin d'accès absolu.

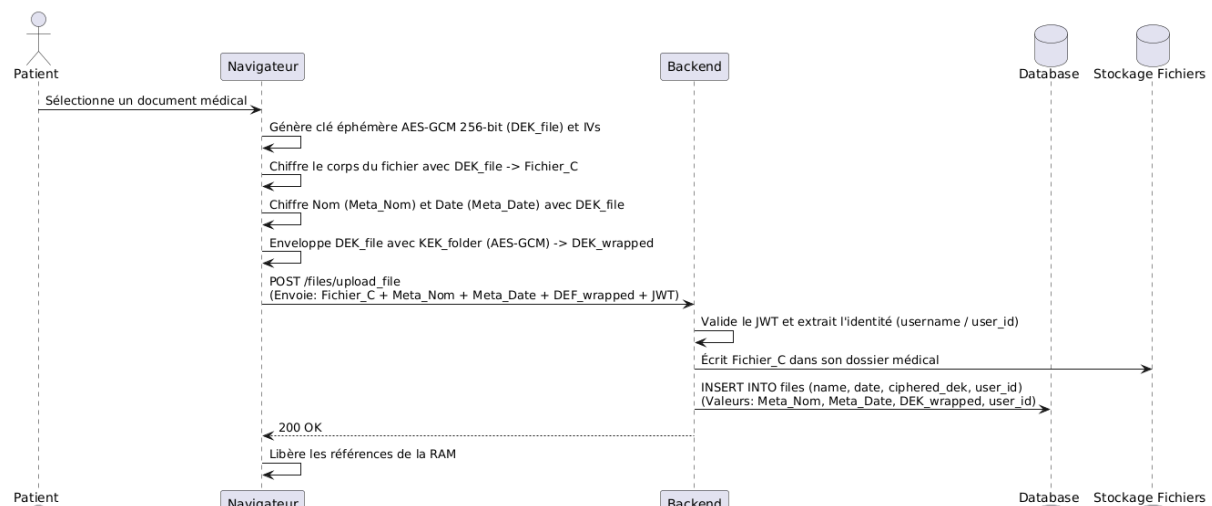


Figure 3-4 : Diagramme de séquence - Upload fichier

3.4 Appairage d'un nouvel appareil

3.4.1 Patient : Passkey lié au hardware

La clé d'accès d'un patient (passkey Windows Hello) est liée au matériel de l'appareil sur lequel elle a été créée. Sur un nouvel appareil physique, cette clé n'existe pas et la connexion est impossible sans étape préalable.

Pour accéder depuis un nouvel appareil, le patient clique sur « Premier accès sur cet appareil » depuis la page de login après il saisit son adresse e-mail et reçoit un lien Keycloak qui l'invite à enregistrer un nouveau passkey Windows Hello sur cet appareil. Une fois l'enregistrement terminé, Keycloak le redirige vers l'application et la connexion OIDC se complète normalement et puis le point critique c'est que l'enregistrement du nouveau passkey ne va pas supprimer ceux des autres appareils.

Une fois qu'il est connecté, le nouvel appareil est en attente d'approbation parce qu'il ne possède pas encore la clé de chiffrement et que l'Appareil A doit approuver l'Appareil B depuis la page de gestion des appareils, en lui transmettant la KEK chiffrée. Ce n'est qu'après cette étape que le patient peut accéder à ses fichiers depuis le nouvel appareil.

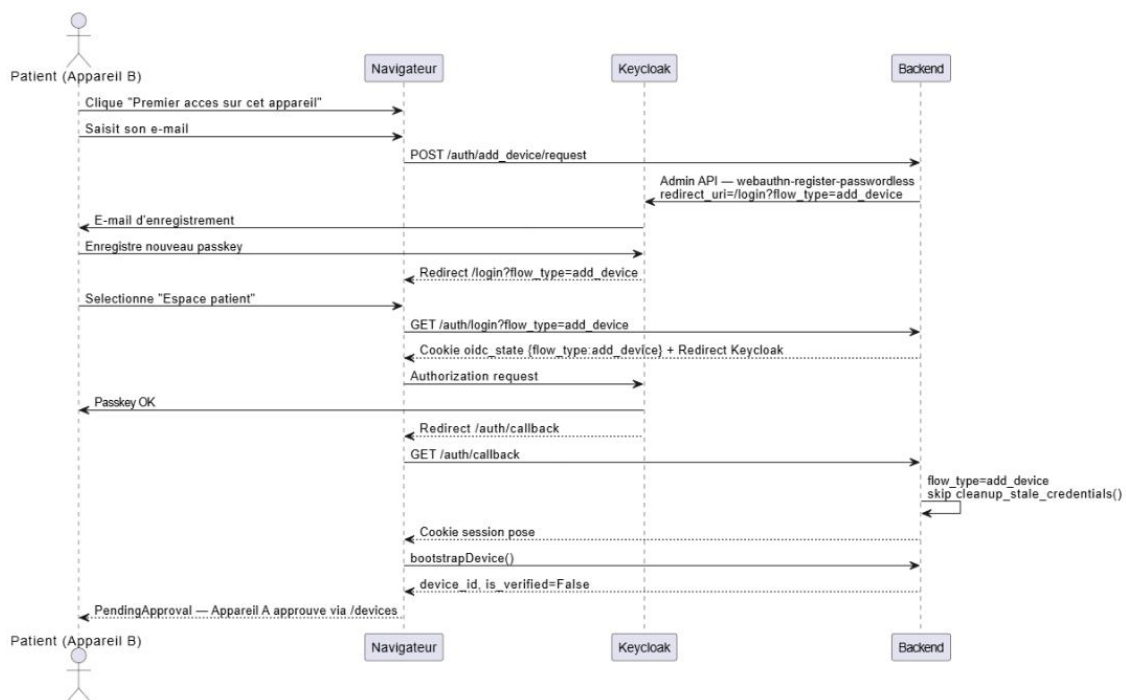


Figure 3-5 : Diagramme de séquence - Multi-device patient

3.4.2 Médecin : Certificat X.509 portable

Le médecin s'authentifie via deux mécanismes : la preuve de possession d'un certificat X.509 (fichier .p12) et un passkey Windows Hello requis par Keycloak. Le fichier .p12 est portable et peut être transporté sur n'importe quel appareil. En revanche, le passkey Windows Hello est lié au matériel de l'appareil, exactement comme pour les patients. Sur un nouvel appareil, le médecin clique sur "Premier accès sur cet appareil" et saisit son adresse e-mail. Il reçoit le même lien Keycloak que les patients pour enregistrer un nouveau passkey. Après l'enregistrement, Keycloak le redirige vers la page de login où le modal de connexion par certificat s'ouvre automatiquement. Le médecin effectue alors la preuve de son certificat .p12 normalement, et la connexion se complète.

On commence par la séquence de passkey d'abord puis après le certificat et ce qui est imposée par l'architecture : Keycloak exige le passkey pour valider la session OIDC, qui n'est déclenchée qu'après la vérification du certificat. Il faut que le passkey existe sur le poste avant de tenter la connexion par certificat et comme pour les patients, le nouvel appareil est en attente d'approbation après la connexion. L'Appareil A doit approuver l'Appareil B en lui transmettant la MEK chiffrée parce que sans ça le médecin ne peut pas déchiffrer les données des patients.

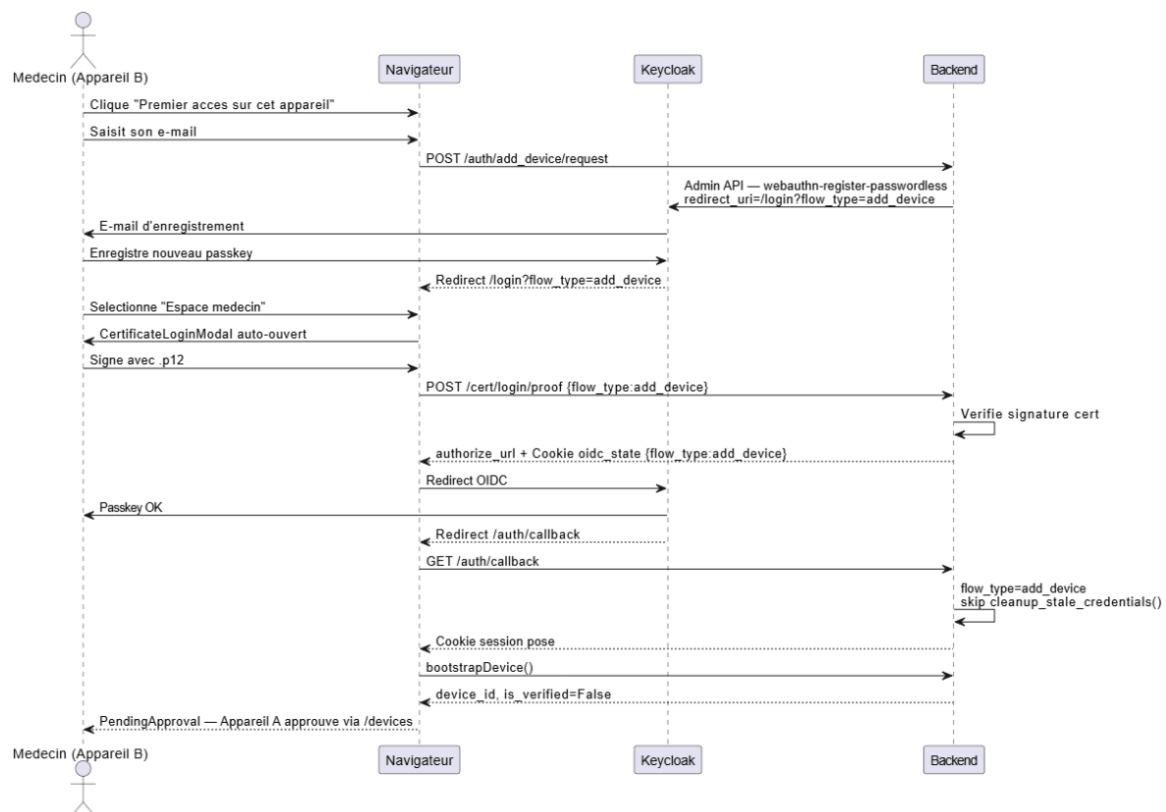


Figure 3-6 : Diagramme de séquence : Multi-device Médecin

3.5 Création d'une relation et accès au dossier patient

Le diagramme de séquence modélise le processus d'établissement d'une relation entre un patient et un médecin. L'opération commence par la consultation du répertoire médical via la route GET /relation/list_doctors, qui retourne la liste des médecins. Une fois le médecin sélectionné, le client déclenche l'initialisation de la liaison via un appel POST /relation/patient_add_doctor. Le backend crée l'enregistrement dans la base de données, activant la relation par défaut tout en laissant l'enveloppe cryptographique vide. En réponse, le serveur transmet au navigateur l'identifiant technique de la relation (relation_id) ainsi que la clé publique maîtresse du médecin ciblé (public_mek).

Le navigateur du patient importe la clé publique reçue, puis procède à l'enveloppement (Key Wrapping) de sa clé de dossier symétrique (KEK_folder), préalablement déchiffrée. Ce chiffrement via l'algorithme RSA-OAEP génère une enveloppe binaire (shared_kek_wrapped) verrouillée, déchiffirable uniquement par la clé privée du médecin. En cas de succès, le client transmet l'enveloppe via un second appel POST /relation/store_kek, le serveur met alors à jour l'enregistrement pour y stocker la clé chiffrée. À l'inverse, si une erreur apparaît, le frontend exécute une procédure de rollback via l'appel POST /relation/patient_remove_doctor, ordonnant au backend de supprimer la relation orpheline. À l'aide de cette KEK, le médecin a désormais la capacité de déchiffrer le contenu du dossier médical de son patient.

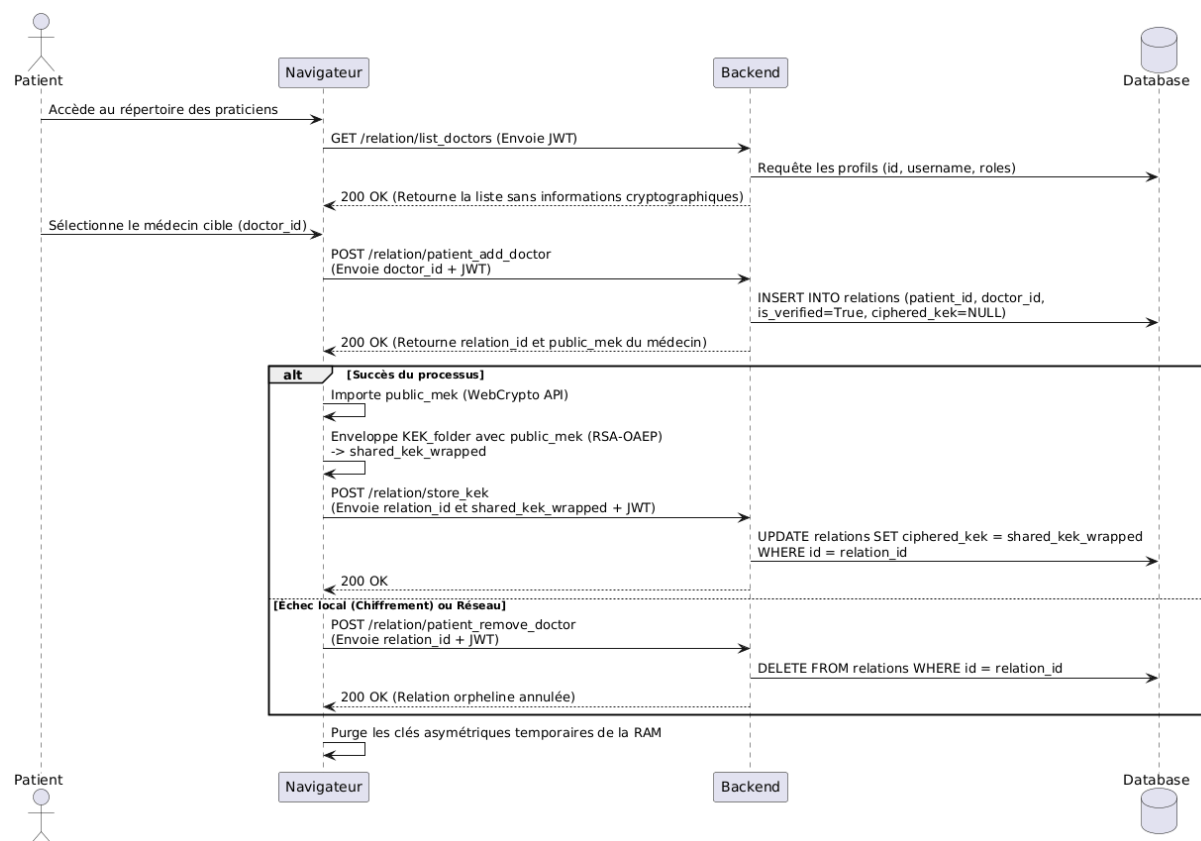


Figure 3-7 : Diagramme de séquence - Création relation

3.6 Flux d'approbation d'une action médecin

Le diagramme de séquence modélise le flux de dépôt d'un fichier, par un médecin, dans le dossier médical de son patient. L'opération débute côté médecin, lorsque le navigateur récupère la clé de dossier chiffrée du patient concerné. Disposant de sa propre clé privée maîtresse, le client du médecin exécute un déchiffrement pour restaurer la clé de dossier symétrique en clair (KEK). Utilisant cette clé, le navigateur chiffre le nouveau fichier : génération d'une clé éphémère, chiffrement des métadonnées, enveloppement cryptographique et encodage Base64. Le dépôt s'effectue ensuite via la route spécifique POST /files/upload_file_for_doctor et inclut l'identifiant du patient (patient_id) en paramètre.

Le fichier chiffré est stocké dans un répertoire temporaire de quarantaine (/temp_storage/{patient_id}/). La demande d'opération est enregistrée dans la base de données. Lors de sa connexion, le patient prend connaissance des documents en attente, via la page de notifications, et appelle la route POST /relation/validate_request pour approuver la requête. C'est lors de cette approbation que le serveur déplace le fichier de la quarantaine vers le dossier médical du patient. Il insère également les métadonnées chiffrées au sein de la base de données.

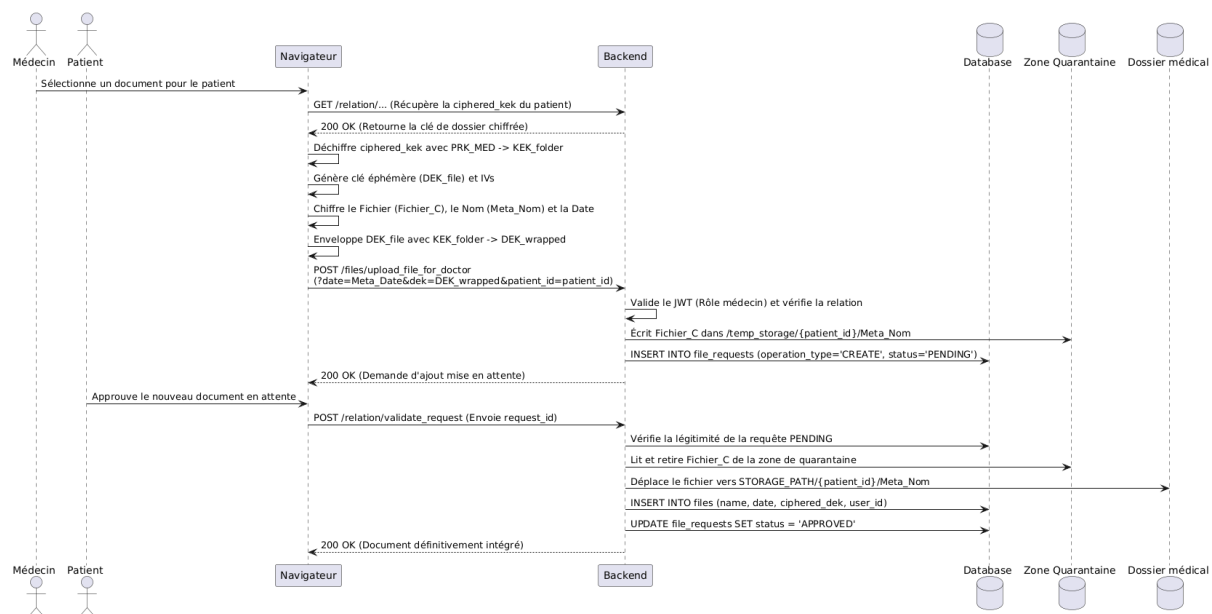


Figure 3-8 : Diagramme de séquence - Upload médecin

Conclusion

Ce projet a permis de mettre en œuvre une application sécurisée dédiée à la gestion de dossiers médicaux entre patients et médecins. L'ensemble des choix techniques réalisés ont pour but de garantir la sécurité des données, même en cas de compromission du serveur. Pour chaque technologie retenue, une analyse comparative a été réalisée pour permettre le choix et les décisions au niveau des différentes technologies.

L'ensemble des technologies retenues forment une bonne architecture répondant aux exigences de confidentialité, d'intégrité et d'authenticité. Ce projet constitue une bonne base qui pourrait, à termes, être utilisable et déployé dans un environnement de production, tout en intégrant aussi Let's Encrypt pour la gestion des certificats HTTPS.

Bibliographie

- [1] « Thomas Ruelle / SécuApp · GitLab », GitLab. Consulté le: 28 mai 2026. [En ligne]. Disponible sur: <https://gitlab.com/Masi-s-Thorue/secuapp>
- [2] K. Team, « Keycloak ». Consulté le: 26 mai 2026. [En ligne]. Disponible sur: <https://www.keycloak.org/>
- [3] « Zitadel | Open Source Alternative to Auth0, Okta ». Consulté le: 26 mai 2026. [En ligne]. Disponible sur: <https://www.opensourcealternative.to/projects/zitadel>
- [4] « OWASP Top Ten Web Application Security Risks | OWASP Foundation ». Consulté le: 26 mai 2026. [En ligne]. Disponible sur: <https://owasp.org/www-project-top-ten/>
- [5] S. P. L, « KeyCloak X509 Certificate Based Login », Medium. Consulté le: 26 mai 2026. [En ligne]. Disponible sur: <https://medium.com/@sangeethapl.sai/keycloak-x509-certificate-based-login-e9101f4a9922>
- [6] « Server Administration Guide ». Consulté le: 26 mai 2026. [En ligne]. Disponible sur: https://www.keycloak.org/docs/latest/server_admin/index.html
- [7] « Set up ZITADEL with Docker Compose | ZITADEL Docs ». Consulté le: 26 mai 2026. [En ligne]. Disponible sur: <https://zitadel.com/docs/self-hosting/deploy/compose>
- [8] « Django », Django Project. Consulté le: 26 mai 2026. [En ligne]. Disponible sur: <https://www.djangoproject.com/>
- [9] « FastAPI - FastAPI ». Consulté le: 26 mai 2026. [En ligne]. Disponible sur: <https://fastapi.tiangolo.com/>
- [10] « Express.js · Node.js web application framework », Express.js. Consulté le: 26 mai 2026. [En ligne]. Disponible sur: <https://expressjs.com/en/>
- [11] « Django vs Express : Quel framework back-end choisir ? » Consulté le: 26 mai 2026. [En ligne]. Disponible sur: <https://www.mobiskill.fr/blog-posts/django-vs-express-quel-framework-back-end-choisir>
- [12] « Django vs FastAPI : Comparaison complète 2026 », elitics.io. Consulté le: 26 mai 2026. [En ligne]. Disponible sur: <https://elitics.io/fr/compare/django-vs-fastapi>
- [13] T. Node, « FastAPI vs Django: A Detailed Comparison in 2025 », Medium. Consulté le: 26 mai 2026. [En ligne]. Disponible sur: <https://medium.com/@technode/fastapi-vs-django-a-detailed-comparison-in-2025-1e70c65b9416>
- [14] « OWASP API Security Project | OWASP Foundation ». Consulté le: 26 mai 2026. [En ligne]. Disponible sur: <https://owasp.org/www-project-api-security/>
- [15] « Web Cryptography API Level 2 ». Consulté le: 4 juin 2026. [En ligne]. Disponible sur: <https://w3c.github.io/webcrypto/>
- [16] « Saving Cryptographic Keys in the Browser », Charles Engelke's Blog. Consulté le: 4 juin 2026. [En ligne]. Disponible sur: <https://blog.engelke.com/2014/09/19/saving-cryptographic-keys-in-the-browser/>
- [17] P. G. D. Group, « PostgreSQL », PostgreSQL. Consulté le: 28 mai 2026. [En ligne]. Disponible sur: <https://www.postgresql.org/>
- [18] « MySQL :: MySQL 8.0 Reference Manual ». Consulté le: 28 mai 2026. [En ligne]. Disponible sur: <https://dev.mysql.com/doc/refman/8.0/en/>
- [19] « MongoDB Documentation - Homepage ». Consulté le: 28 mai 2026. [En ligne]. Disponible sur: <https://www.mongodb.com/docs/>
- [20] « nginx ». Consulté le: 26 mai 2026. [En ligne]. Disponible sur: <https://nginx.org/>
- [21] « Welcome! - The Apache HTTP Server Project ». Consulté le: 26 mai 2026. [En ligne]. Disponible sur: <https://httpd.apache.org/>
- [22] H. Team, « Apache vs nginx: choisir le bon serveur Web pour votre pile | Hostwinds », Hostwinds Blog. Consulté le: 26 mai 2026. [En ligne]. Disponible sur: <https://www.hostwinds.fr/blog/apache-vs-nginx-web-servers>

- [23] « NGINX vs. Apache : comparaison des architectures et des possibilités de configuration et d'extension », ionos Digital Guide. Consulté le: 26 mai 2026. [En ligne]. Disponible sur: <https://www.ionos.fr/digitalguide/serveur/know-how/nginx-vs-apache/>
- [24] Olivier, « Let's Encrypt: les avantages et limites d'un SSL gratuit », Ex2 - Hébergeur Web vert spécialisé WordPress. Consulté le: 26 mai 2026. [En ligne]. Disponible sur: <https://www.ex2.com/tutoriels/lets-encrypt-les-avantages-et-limites-dun-ssl-gratuit/>
- [25] O. Owolafe et A. Alabi, *Comparative Analysis of Wazuh, Graylog, and Elk Stack for Cyber Threat Detection and Response*. 2024, p. 193. doi: 10.20533/ICITST.2024.0015.
- [26] athanielx, « Wazuh vs ELK SIEM », r/cybersecurity. Consulté le: 26 mai 2026. [En ligne]. Disponible sur: https://www.reddit.com/r/cybersecurity/comments/1gf6euj/wazuh_vs_elk_siem/
- [27] « Which One's Your Perfect Fit: Elastic Stack vs Splunk vs Wazuh ». Consulté le: 26 mai 2026. [En ligne]. Disponible sur: <https://www.linkedin.com/pulse/which-ones-your-perfect-fit-elastic-stack-vs-splunk-hassain-k-k-51bfc>
- [28] « Chiffrement RSA », *Wikipédia*. 28 février 2026. Consulté le: 28 mai 2026. [En ligne]. Disponible sur: https://fr.wikipedia.org/w/index.php?title=Chiffrement_RSA&oldid=233697994
- [29] K. Moriarty, B. Kaliski, J. Jonsson, et A. Rusch, « PKCS #1: RSA Cryptography Specifications Version 2.2 », Internet Engineering Task Force, Request for Comments RFC 8017, nov. 2016. doi: 10.17487/RFC8017.
- [30] E. Barker, L. Chen, A. Roginsky, A. Vassilev, R. Davis, et S. Simon, « Recommendation for Pair-Wise Key-Establishment Using Integer Factorization Cryptography », National Institute of Standards and Technology, NIST Special Publication (SP) 800-56B Rev. 2, mars 2019. doi: 10.6028/NIST.SP.800-56Br2.
- [31] R. Housley et J. Schaad, « Advanced Encryption Standard (AES) Key Wrap Algorithm », Internet Engineering Task Force, Request for Comments RFC 3394, oct. 2002. doi: 10.17487/RFC3394.
- [32] « Cryptographic Storage - OWASP Cheat Sheet Series ». Consulté le: 28 mai 2026. [En ligne]. Disponible sur: https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.html
- [33] E. Barker, « Recommendation for Key Management: Part 1 – General », National Institute of Standards and Technology, NIST Special Publication (SP) 800-57 Part 1 Rev. 5, mai 2020. doi: 10.6028/NIST.SP.800-57pt1r5.
- [34] « WebAuthn », *Wikipédia*. 15 juin 2025. Consulté le: 28 mai 2026. [En ligne]. Disponible sur: <https://fr.wikipedia.org/w/index.php?title=WebAuthn&oldid=226525301>

Table des illustrations

<i>Figure 3-1 : Diagramme de séquence - Inscription patient</i>	<i>13</i>
<i>Figure 3-2 : Diagramme de séquence - Génération certificats</i>	<i>14</i>
<i>Figure 3-3 : Diagramme de séquence - Inscription médecin</i>	<i>15</i>
<i>Figure 3-4 : Diagramme de séquence - Upload fichier</i>	<i>16</i>
<i>Figure 3-5 : Diagramme de séquence - Multi-device patient</i>	<i>17</i>
<i>Figure 3-6 : Diagramme de séquence : Multi-device Médecin</i>	<i>18</i>
<i>Figure 3-7 : Diagramme de séquence - Création relation.....</i>	<i>19</i>
<i>Figure 3-8 : Diagramme de séquence - Upload médecin.....</i>	<i>20</i>